tmux documentation

tmx-ssh-dispatch

Revision: 1 **Last modified:** 2026-05-22T00:00:00Z **Authority:** vasic-digital tmux project **Maintainer:** milosvasic **Scope:** §11.4.18 script companion doc for `scripts/tmx-ssh-dispatch.sh` (generated from `scripts/tmx-ssh-dispatch.sh.template`)

Purpose

Remote-side `command="…"` dispatcher referenced from `~/.ssh/authorized_keys`. Translates whatever the SSH client passes after `ssh <host>-tmx <…>` (kernel exposes it as `$SSH_ORIGINAL_COMMAND`) into one of three outcomes:

1. **empty** → exec `bash -l` (interactive login; the operator's normal `.bashrc` then sources `tmx-shell-init.sh` for the regular interactive prompt flow).
2. **valid tmx session name** (POSIX `case`-matching `^[A-Za-z0-9_.-]{1,64}$`) → `tmx attach -t NAME` if the session exists, else `tmx new -s NAME -c <last-pwd>` where `<last-pwd>` comes from `tmx-state recall NAME`.
3. **anything else** (multi-word command, shell injection attempt, oversized token) → write a one-line rejection to stderr and `exit 1`.

This is the per-host SSH key's only permitted action surface — operators who need a real shell over the same host use a different key without the `command=` restriction.

Usage

The script is invoked by sshd, never by hand. The relevant authorized_keys line on the remote machine looks like:

```
command="/path/to/project/scripts/tmx-ssh-dispatch.sh",no-port-forwarding,
```

`scripts/tmx-ssh-install.sh` installs that line idempotently — see `docs/scripts/tmx-ssh-install.md`.

Operator-facing usage from the client side:

```
ssh <host>-tmx              # empty SSH_ORIGINAL_COMMAND → bash
        -l
ssh <host>-tmx work         # valid name → attach/create session
        'work'
ssh <host>-tmx 'foo bar'    # rejected; exit 1; stderr [tmx-ssh-
        dispatch]…
```

Inputs

Environment variables (set by sshd)

Var	Meaning
`SSH_ORIGINAL_COMMAND`	The argv tail the client passed after `ssh <alias>`. Empty when the client omitted any command.
`HOME`	Used as the cwd fallback when `tmx-state recall` returns nothing or the recalled directory no longer exists.
`TMX_DISPATCH_TEST`	**Test-mode only.** When set, replaces every `exec` with a logging stub that prints `[would exec: …]` to stdout. Off in production.

Filesystem inputs

- `tmx-state` binary — looked up first on `PATH`, then at `<project>/scripts/tmx-state-bin`. If both are missing, the dispatcher falls back to no-cwd-restore (still works — just lands in `$HOME`).
- `tmx` wrapper — looked up first on `PATH`, then at `<project>/scripts/tmx`. If both are missing, the dispatcher emits a stderr error and exits 1.

Outputs

- **stdout:** nothing in production; in `TMX_DISPATCH_TEST=1` mode it prints the `[would exec: …]` line that captures which branch was taken.
- **stderr:** one-line rejection messages on bad input; nothing on success.
- **exit code:** 0 on the `exec` paths (only meaningful when test-mode short-circuits the actual exec); 1 on rejection.

Side-effects

None apart from the `exec` that replaces the dispatcher's process image with either `bash -l` (login path) or the tmx wrapper (attach/new path). The script never writes to disk, never edits state, never creates background processes.

Dependencies

- POSIX `/bin/sh` (script parses clean under sh, dash, bash 3.2/5, mksh).
- `tmx` wrapper — required for the session-attach path; the dispatcher emits a clear error if absent.
- `tmx-state` binary — optional; dispatcher gracefully degrades to "no cwd restore" if absent.

Cross-references

- Spec: `docs/superpowers/specs/2026-05-22-tmx-shell-session-resume-design.md` §4.C, §5.2, §6 (edge cases 3 + 9 + 13).
- Operator guide (installation + usage): `docs/guides/tmx-ssh-dispatch.md` (to be generated by PWU P8).

- Sibling script: `scripts/tmx-ssh-install.sh` (installer) and its companion `docs/scripts/tmx-ssh-install.md`.
- Wrapper: `scripts/tmx.template` (the `tmx new` / `tmx attach` end the dispatcher exec's into).
- Governance: §11.4.18 (script docs), §11.4.44 (revision header), §11.4.67 (POSIX-parseable shell), §11.4.50 (deterministic consistency — dispatcher behaves identically across N iterations because no time/random inputs).

Debugging

When `ssh <alias> <session>` misbehaves the diagnostic ladder is:

1. **Client side:** `ssh -v <alias> work 2>&1 | head -40` — the verbose log shows whether the key matched and whether the `command=` was applied.
2. **Remote sshd logs:** `journalctl -u ssh.service` (Linux systemd) or `tail -F /var/log/auth.log` (Debian/Ubuntu) or `log show --predicate 'process == "sshd"' --last 5m` (macOS) shows server-side decisions about which key and command line matched.
3. **Dispatcher dry-run on the remote:** `SSH_ORIGINAL_COMMAND=work TMX_DISPATCH_TEST=1 sh /path/to/tmx-ssh-dispatch.sh` reveals which branch the dispatcher would take without actually exec'ing tmux. This is the primary anti-bluff smoke test referenced by §11.4.50 and is the same probe runtime test 22 (`22_ssh_dispatch_local.sh`) uses.
4. **Reject probe (no key needed remotely):** `ssh <alias> tmx-install-verify-token-XXXX` should produce a `[tmx-ssh-dispatch]…` stderr line and exit 1. If it produces something else, the `authorized_keys command="…"` did not apply — likely a path mismatch.

Last verified

2026-05-22 — sh -n clean on substituted template; four anti-bluff smoke tests (empty / valid-name / multi-word / shell-injection) all produced the expected branch / stderr / exit code on macOS. Full real-host verification deferred to P9 release verification per spec §10.